School of Computing
Queen's University

CISC235

Winter 2026

Assignment 3

Deadline: Mar 11, 2026, 11:59 PM

"Hashing – Open addressing"

Implement Map using open addressing with linear probing
You are given a "supposed to be a complete implementation source code" of a hashing class called "Map." This hashing class is based on linear probing. Some parts have been taken out. You are requested in this assignment to complete these parts that are missing. To help, the missing parts belong to 13 different methods as numbered in the source code. Each empty rectangle requires some code to run in agreement with the rest of the given code. Without amending the given code, add the missing parts at the location of each given empty rectangle.

Below is a run to test your work. It is recommended that you use the same input to test your work. It is a run for a file that contains the given "Map" class. The file named "235-W26-Assignment3.py." The run contains 3 parts: 1) the first line to run the Python program, 2) lines 2 -12, inclusive, stands for interacting with the program by entering the requested data, and 3) line number 13 to end, contains the output of your program. Your test output should match if it runs correctly.

```
command>python 235-W26-Assignment3.py
Enter five names and ages
Enter a name: Anderson
Enter an age: 31
Enter a name: Cook
Enter an age: 29
Enter a name: Cook
Enter an age: 129
Enter a name: Smith
Enter an age: 30
Enter a name: Lewis
Enter an age: 29
Entry set in map: (['Anderson', 31], ['Cook', 29], ['Cook', 129], ['Lewis', 29], ['Smith', 30])
The age for Lewis is 29
Is Smith in the map? True
Is Johnson in the map? False
Is value 30 in the map? True
Is value 33 in the map? False
Is age 33 in the map? False
All values for Cook? (29, 129)
keys are ['Anderson', 'Cook', 'Cook', 'Lewis', 'Smith']
values are ['Anderson', 'Cook', 'Cook', 'Lewis', 'Smith']
The map is [['Anderson', 31], ['Cook', 29], ['Cook', 129], None, ['Lewis', 29], None, None, (None,
        None)]
The map is [None, None, None, None, None, None, None, None]
```

How to Submit:

Write the requested program in Python. Submit it using the drop box of Assignment 3. You should submit: 1) the source code file, and 2) a screen shot of successful run such as the one shown above in this assignment.
Please include the following sentence in the top documentation of your program (Source code):
I, _____ (student's printed name) hereby declare that the attached assignment has been completed and submitted as required by myself as an individual.

========== Beginning of the source code of class 'Map'==========

```
# Define the default hash-table size
DEFAULT_INITIAL_CAPACITY = 4

# Define default load factor
DEFAULT_MAX_LOAD_FACTOR = 0.75

# Define the maximum hash-table size to be 2 ** 30
MAXIMUM_CAPACITY = 2 ** 30

class Map:
    def __init__(self, capacity = DEFAULT_INITIAL_CAPACITY,
                 loadFactorThreshold = DEFAULT_MAX_LOAD_FACTOR):
        # Current hash-table capacity. Capacity is a power of 2
        self.capacity = capacity

        # Specify a load factor used in the hash table
        self.loadFactorThreshold = loadFactorThreshold

        # Create a list of empty buckets
        self.table = []
        for i in range(self.capacity):
            self.table.append(None) # Use open addressing

        self.size = 0 # Initialize map size

    # (1) Add an entry (key, value) into the map
    def put(self, key, value):
        [                                                  ]

        # Use open addressing
        [                                                  ]

        # Add an entry (key, value) to hashTable[index]
        [                                                  ]
```

```
# (2) Remove the entry for the specified key
def remove(self, key):
    i = self.getHash(hash(key))

    # Use open addressing


    # Remove the first entry that matches the key from a bucket


# (3) Search for key in an entry. Return true if the specified key is in the map
def containsKey(self, key):


# (4) Search for a value in an entry. Return true if this map contains the specified value
def containsValue(self, value):


# (5) List enteries. Return a set of entries in the map
def items(self):


# (6) Search. Return the first value that matches the specified key
def get(self, key):


# (7) Search. Return all values for the specified key in this map
def getAll(self, key):
```

```
# (8) Return a set consisting of the keys in this map
def keys(self):

# (9) Return a set consisting of the values in this map
def values(self):

# (10) Remove all of the entries from this map
def clear(self):

# (11) Return the number of mappings in this map
def getSize(self):

# (12) Return true if this map contains no entries
def isEmpty(self):

# Rehash the map
def rehash(self):
    temp = self.items() # Get entries
    self.capacity *= 2 # Double capacity
    self.table = [] # Create a new hash table
    self.size = 0 # Clear size
    for i in range(self.capacity):
```

```
            self.table.append(None)

        for entry in temp:
            self.put(entry[0], entry[1]) # Store to new table

    # (13) Return the entries as a string
    def toString(self):
        [redacted]

    # Return a string representation for this map
    def setLoadFactorThreshold(self, threshold):
        self.loadFactorThreshold = threshold

    # Return the hash table as a string
    def getTable(self):
        return str(self.table)

    def supplementalHash(self, h):
        h ^= (h >> 20) ^ (h >> 12)
        return h ^ (h >> 7) ^ (h >> 4)

    def getHash(self, hashCode):
        return self.supplementalHash(hashCode) & (self.capacity - 1)

def main():
    # Create a map
    map = Map()

    print("Enter five names and ages")
    for i in range(5):
        name = input("Enter a name: ")
        age = int(input("Enter an age: "))
        map.put(name, age) # Add (name, age) to map

    print("Entry set in map: " + str(map.items()))
    print("The age for Lewis is " + str(map.get("Lewis")))
    print("Is Smith in the map? " + str(map.containsKey("Smith")))
    print("Is Johnson in the map? " +
        str(map.containsKey("Johnson")))
    print("Is value 30 in the map? " + str(map.containsValue(30)))
    print("Is value 33 in the map? " + str(map.containsValue(33)))
    print("Is age 33 in the map? " + str(map.containsValue(33)))
    print("All values for Cook? " + str(map.getAll("Cook")))
    print("keys are " + str(map.keys()))
    print("values are " + str(map.values()))

    map.remove("Smith") # Remove Smith from map
    print("The map is " + map.getTable())

    map.clear()
    print("The map is " + map.getTable())

main()
```

========== End of the source code of class 'Map' ==========